SLM PLATFORM
PRODUCT GUIDE

THE COMPLETE STRATEGY, IMPLEMENTATION,
AND ADVANCED ANALYTICS HANDBOOK

SLM Platform

Product Guide for ML Engineers

Version 1.0 | 2026 Edition

Optimized for NVIDIA DGX Spark

Confidential & Proprietary

Table of Contents

1	**Getting Started**	Project creation and platform overview
2	**Data Ingestion**	File uploads, HuggingFace, Kaggle, and URL connectors
3	**Data Cleaning & PII Redaction**	Deduplication, quality filtering, and privacy
4	**Gold Dataset Builder**	Curating ground-truth evaluation sets
5	**Synthetic Data Generation**	AI-powered Q&A pair generation
6	**Training & Fine-Tuning**	LoRA configuration, optimizers, and live telemetry
7	**Evaluation & LLM-as-a-Judge**	Benchmarking, radar analytics, side-by-side arena
8	**Compression & Quantization**	4-bit, 8-bit weight optimization
9	**Export & Deployment**	GGUF, ONNX, and HuggingFace Hub publishing
A	**Architecture Reference**	API endpoints and system design

Chapter 1

Getting Started

The SLM Platform is a production-grade orchestration engine for fine-tuning, evaluating, and deploying Small Language Models. It is designed for ML engineers who need granular control over every stage of the model lifecycle, from raw data ingestion through to quantized GGUF export.

The platform runs as a decoupled frontend + backend system. The React dashboard communicates with a FastAPI backend that orchestrates all compute-intensive tasks via PyTorch and HuggingFace Transformers.

System Requirements

1. **Hardware**
   NVIDIA GPU with 16GB+ VRAM (DGX Spark recommended). 32GB+ system RAM.
2. **Software**
   Python 3.11+, Node.js 20+, CUDA 12.x toolkit.
3. **Dependencies**
   PyTorch 2.x, HuggingFace Transformers, PEFT, bitsandbytes, vLLM (optional).

Launching the Platform

1. **Start the Backend**
   cd backend && python -m uvicorn app.main:app --reload --port 8000
2. **Start the Frontend**
   cd frontend && npm run dev
3. **Open the Dashboard**
   Navigate to http://localhost:5173 in your browser.
4. **Create a Project**
   Click 'Create New Project', name it (e.g. 'Customer Support SLM'), and enter the project.

> **TIP**
> The sidebar shows all 10 pipeline stages. Work through them left-to-right for a complete fine-tuning workflow.

Chapter 2

Data Ingestion

The Data Ingestion panel is the entry point for all training data. It supports four source types, accessible via tabs at the top of the panel.

Local File Upload

1. **Select 'Local Files' Tab**
   This is the default view. You will see a drag-and-drop upload zone.
2. **Upload Documents**
   Drag files into the zone or click 'Choose Files'. Supported formats: PDF, DOCX, TXT, Markdown, CSV, JSON, JSONL.
3. **Process Documents**
   After upload, each file appears in the Ingested Documents table with status 'pending'. Click the gear icon to parse and extract text.

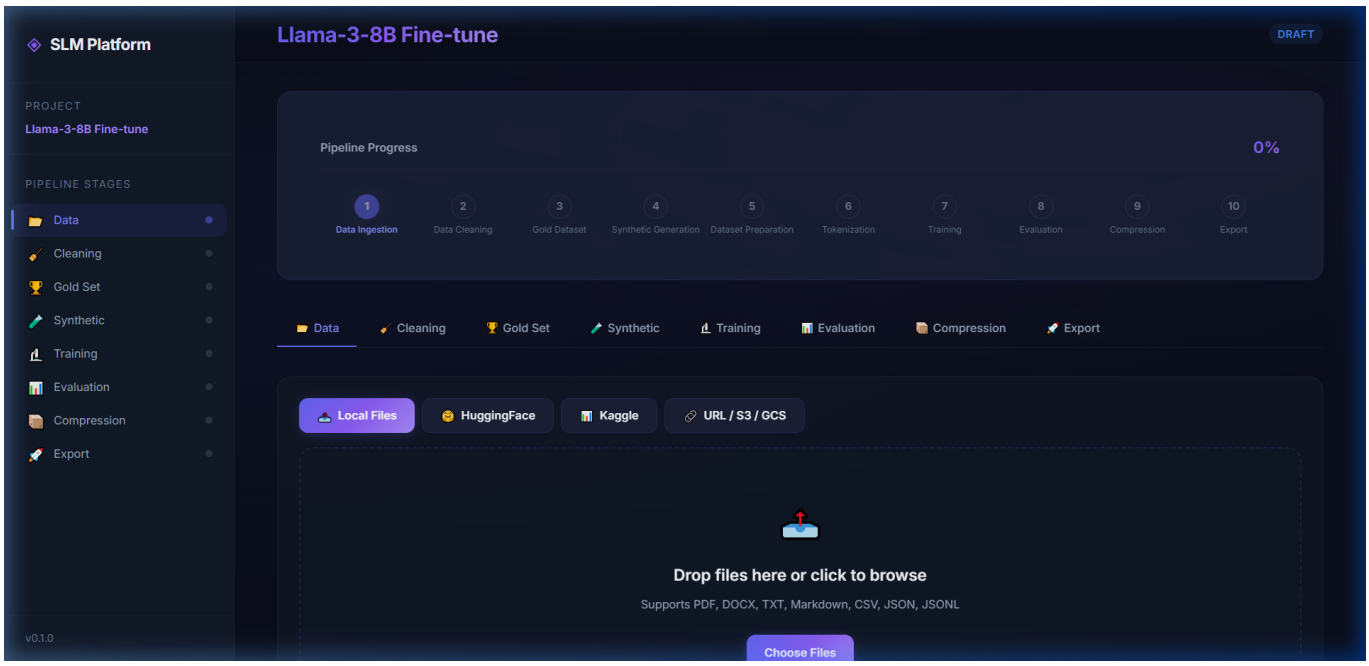

Figure 2.1 -- Data Ingestion panel with Local Files tab and source connectors

HuggingFace Hub Connector

1. **Click the 'HuggingFace' Tab**

   The form switches to show a Dataset Identifier field and Split selector.

2. **Enter Dataset ID**

   Type the HuggingFace dataset path, e.g. 'tatsu-lab/alpaca', 'squad', or 'openai/gsm8k'.

3. **Select Split**

   Choose train, test, or validation from the dropdown.

4. **Set Max Samples (Optional)**

   Limit the number of rows to import for faster iteration.

5. **Click 'Import from HuggingFace'**

   The backend downloads and converts the dataset to JSONL format.

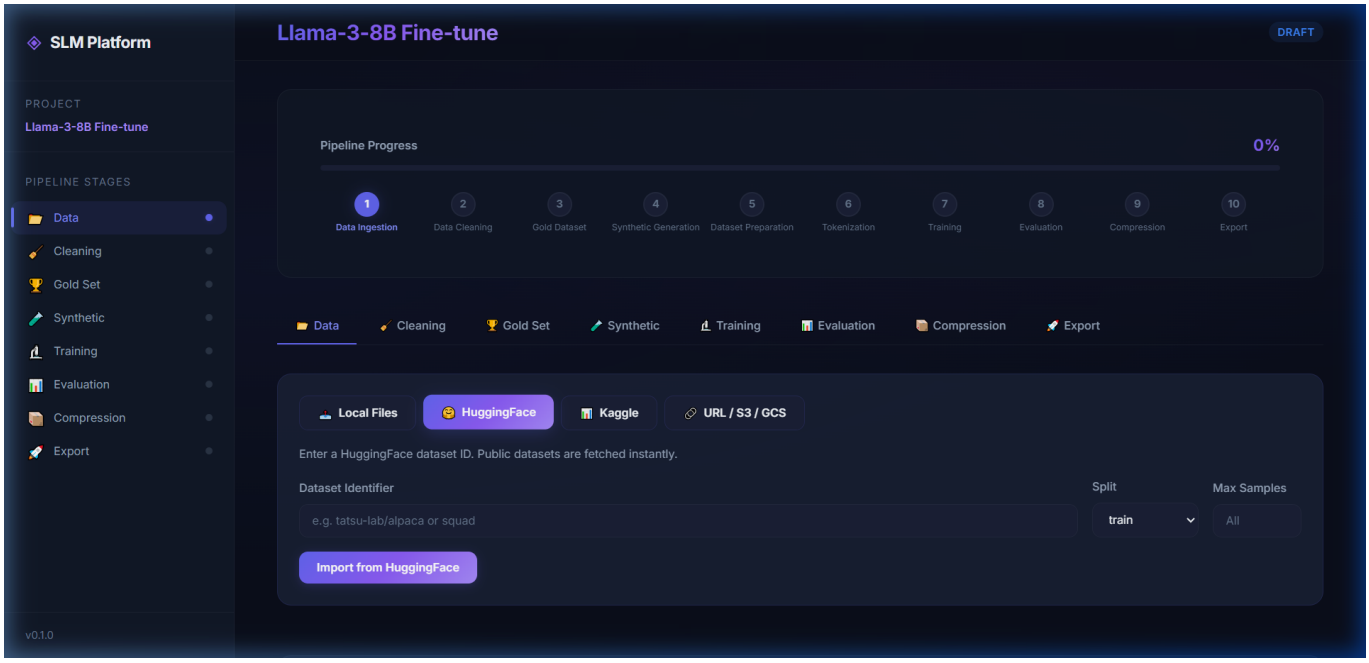

Figure 2.2 -- HuggingFace Hub connector with dataset identifier and split selector

Kaggle Connector

1. **Click the 'Kaggle' Tab**

   Requires KAGGLE_USERNAME and KAGGLE_KEY environment variables on the server.

2. **Enter Dataset Slug**

   Use the format 'username/dataset-name' as shown on Kaggle.

3. **Click 'Import from Kaggle'**

   The dataset is downloaded and converted to JSONL.

URL / S3 / GCS Connector

1. **Click the 'URL / S3 / GCS' Tab**

   For importing data from any direct HTTP link or cloud storage URI.

2. **Paste the URL**

   Supports direct links to .csv, .json, or .jsonl files, as well as S3 and GCS URIs.

3. **Click 'Import from URL'**

   The file is fetched and stored in the project's data directory.

> **TIP**
>
> All imported datasets appear in the same Ingested Documents table. The 'Source' column shows where each file came from (e.g. huggingface:tatsu-lab/alpaca).

Chapter 3

Data Cleaning & PII Redaction

Raw data often contains duplicates, noise, and sensitive personally identifiable information (PII). The Cleaning module provides automated tools to sanitize your dataset before training.

Step-by-Step

1. **Navigate to 'Cleaning'**
   Click the Cleaning tab in the left sidebar.
2. **Select a Dataset**
   Choose the raw dataset you want to clean from the dropdown.
3. **Configure Chunking**
   Set the chunk size (characters) and overlap. Smaller chunks work better for Q&A; larger chunks for summarization.
4. **Enable Deduplication**
   Check 'Exact Dedup' for hash-based removal, or 'Semantic Dedup' for embedding-based near-duplicate detection.
5. **Enable PII Redaction**
   Automatically masks emails, phone numbers, SSNs, and other PII patterns.
6. **Run Cleaning Job**
   Click the action button. Progress is shown in real-time.

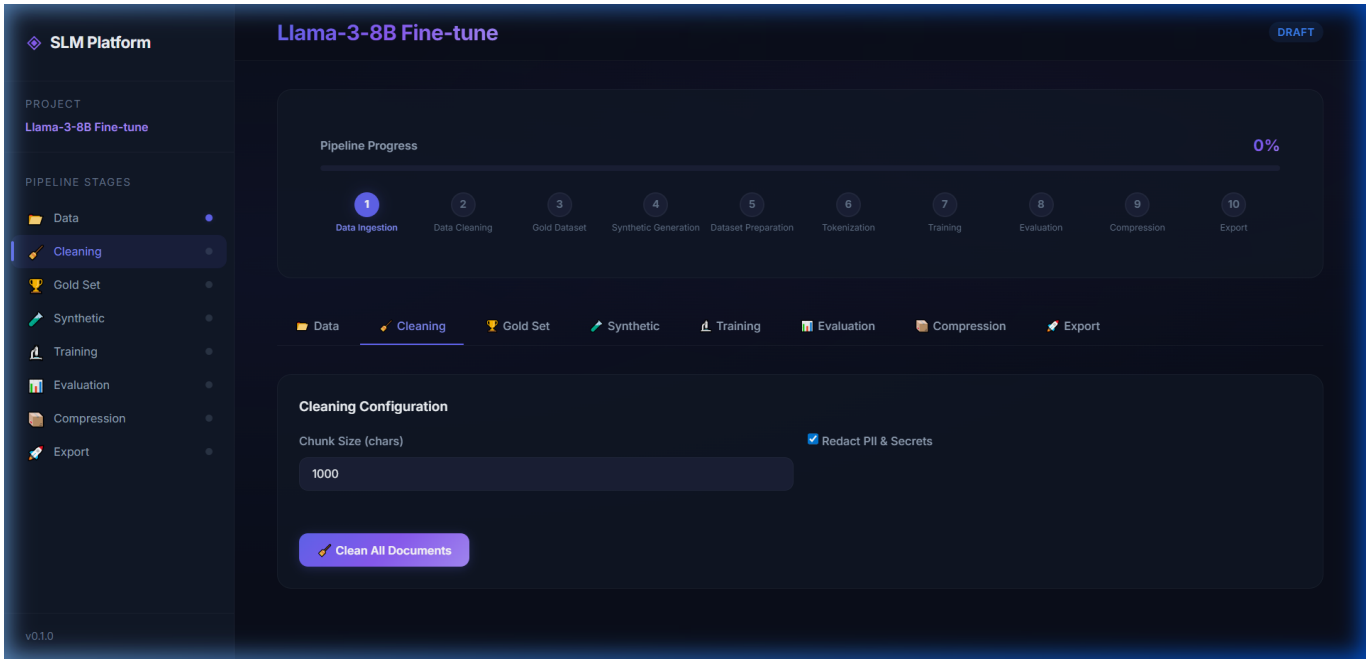

Figure 3.1 -- Data Cleaning panel with chunking strategy and PII redaction options

TIP

PII redaction uses regex patterns and NER models to detect sensitive data. Always review a sample of cleaned records before training.

Chapter 4

Gold Dataset Builder

Gold datasets are hand-curated, high-quality Q&A pairs that serve as the ground truth for evaluation. A strong gold set is critical for measuring model quality after fine-tuning.

Step-by-Step

1. **Navigate to 'Gold Set'**
   Click the Gold Set tab in the sidebar.
2. **Add Q&A Pairs Manually**
   Enter a question in the 'Question' field and the expected answer in the 'Answer' field.
3. **Set Difficulty Level**
   Tag each pair as Easy, Medium, or Hard to enable stratified evaluation.
4. **Import from Existing Data**
   You can also auto-generate gold set candidates from your cleaned dataset.
5. **Review and Approve**
   Each pair has an approve/reject button. Only approved pairs are used for evaluation.

Real-World Example

If building a customer support SLM, your Gold Set should contain exact expert answers:

Q: What is the refund policy for annual subscriptions cancelled after 30 days?
A: Annual subscriptions cancelled after 30 days are non-refundable, but the account remains active until the end of the billing cycle. (See Terms section 4.2)

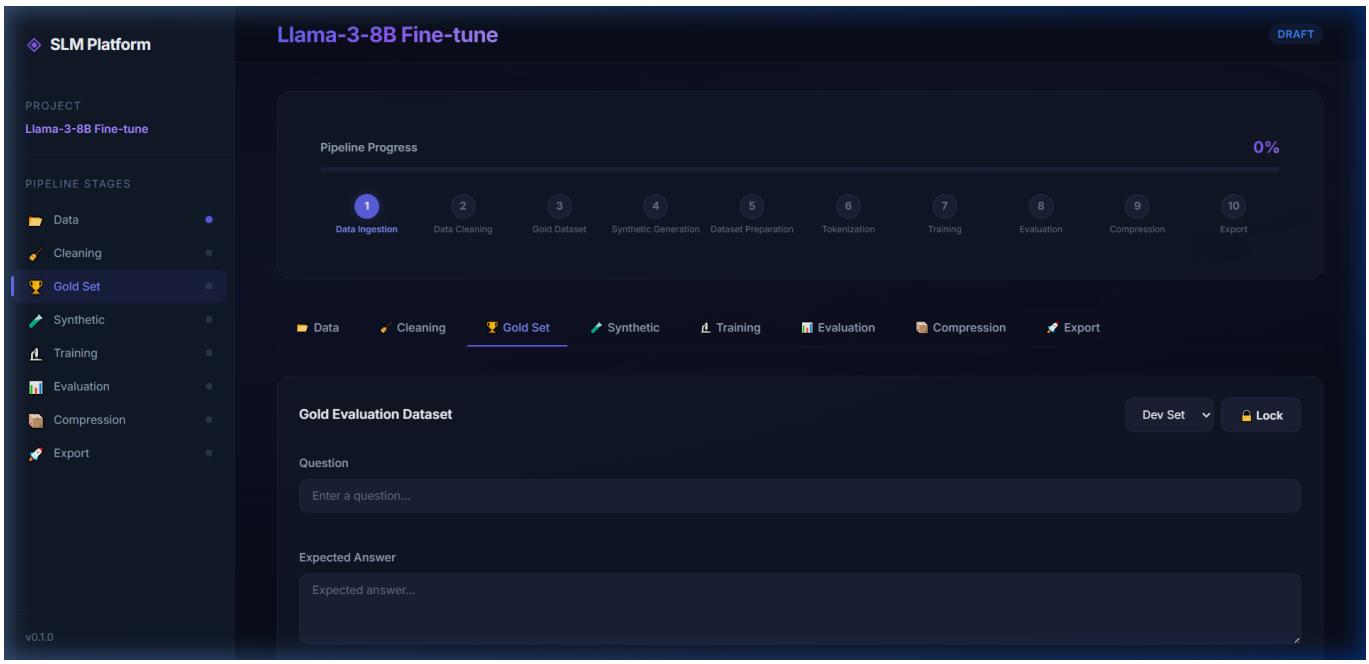

Figure 4.1 -- Gold Dataset Builder interface

Chapter 5

Synthetic Data Generation

When manual data creation is too slow, use the Synthetic Generation module. A large Teacher model (e.g. Llama-3-70B) reads your source documents and automatically generates diverse, high-quality Q&A pairs.

Step-by-Step

1. **Navigate to 'Synthetic'**
   Click the Synthetic tab in the sidebar.
2. **Select Teacher Model**
   Choose a strong model from the dropdown (e.g. meta-llama/Meta-Llama-3-70B-Instruct).
3. **Configure Generation**
   Set the number of pairs to generate, temperature, and context window size.
4. **Run Generation**
   Click the generate button. The Teacher model processes your documents and creates Q&A pairs.
5. **Review Output**
   Generated pairs appear in a table. Approve, edit, or reject each one before adding to your training set.

Local Deployment via Ollama

The platform supports local inference via Ollama. If you have a local model (e.g. Llama-3 or Mixtral) running on localhost:11434, you can use it as the Teacher model to ensure no data leaves your network.

How Generation Works

1. Source Text from your raw documents:

"The HTTP 429 Too Many Requests response status code indicates the user has sent too many requests in a given amount of time ('rate limiting')."

2. Teacher Output (Generated Training Pair):

Q: What does an HTTP 429 status code mean?
A: It means Too Many Requests, indicating the user has hit a rate limit.

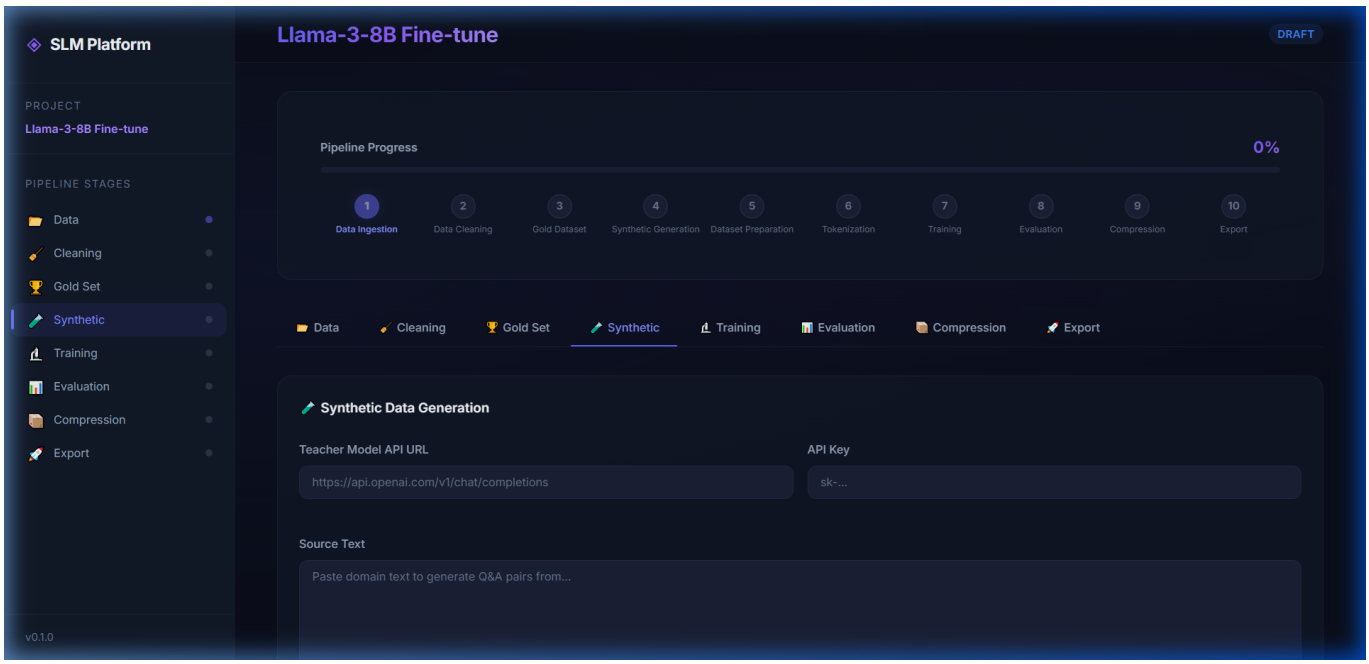

Figure 5.1 -- Synthetic Data Generation panel with Teacher model configuration

TIP

Use a high temperature (0.8-1.0) for diverse question styles. Use low temperature (0.2-0.4) for factual, precise pairs.

Chapter 6

Training & Fine-Tuning

The Training Dashboard provides full control over Parameter-Efficient Fine-Tuning (PEFT) using LoRA/QLoRA. It includes real-time WebSocket telemetry for monitoring loss, learning rate, and GPU utilization.

Configuring a Training Run

1. **Navigate to 'Training'**
   Click the Training tab in the sidebar.
2. **Create New Experiment**
   Click 'New Experiment'. Name it descriptively (e.g. 'v1-customer-support-lora').
3. **Set Base Model**
   Enter the HuggingFace model ID (e.g. meta-llama/Meta-Llama-3-8B).
4. **Configure LoRA Parameters**

- Rank (r): Controls adapter capacity. Higher = more expressive but more VRAM. Start with 16.
- Alpha: Scaling factor. Set to 2x rank (e.g. Alpha=32 for Rank=16).
- Target Modules: q_proj, v_proj (default). Add k_proj, o_proj for deeper adaptation.
- LoRA Dropout: 0.05-0.1 for regularization.

5. **Select Optimizer**
   Paged AdamW 8-bit is recommended for VRAM efficiency on consumer GPUs. Standard AdamW for DGX-class hardware.
6. **Set Learning Rate Schedule**
   Cosine with warmup is the default. Linear decay works for shorter runs.
7. **Enable Flash Attention 2**
   Check this box if your GPU supports it (Ampere+). Reduces memory usage by 2-4x.
8. **Launch Training**
   Click 'Start Training'. The live dashboard will appear.

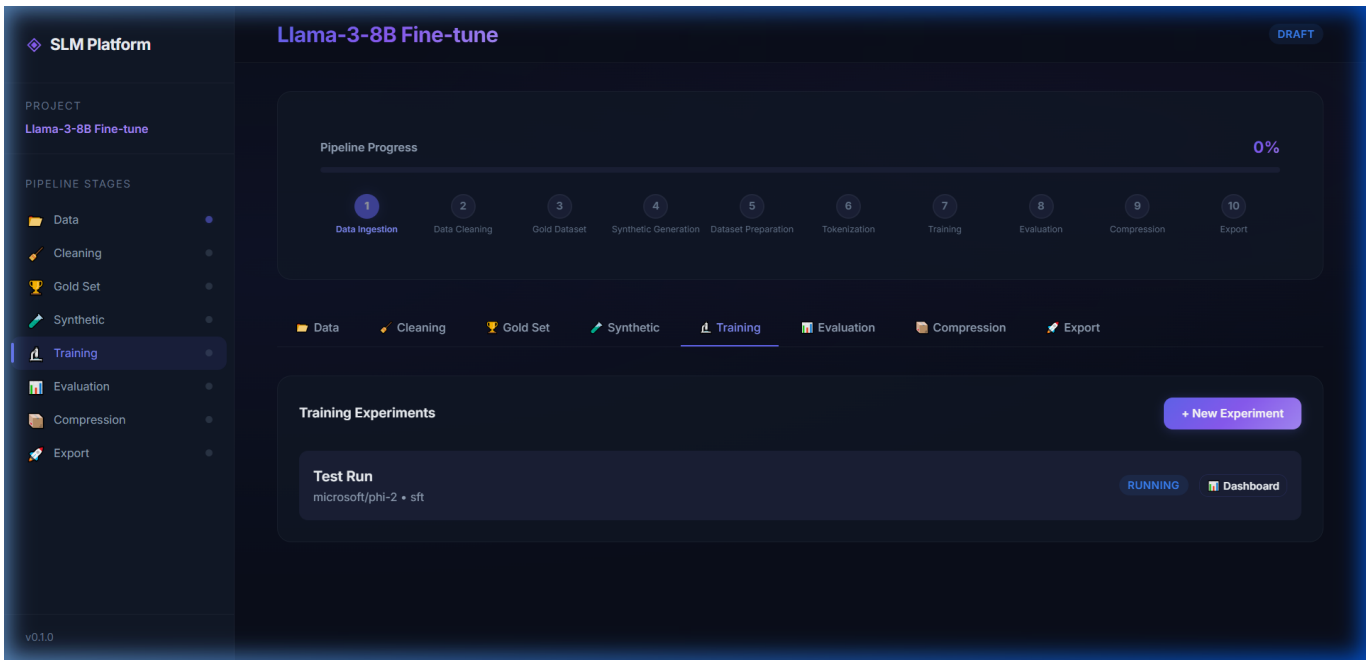

Figure 6.1 -- Training Dashboard with experiment configuration

Live Telemetry Dashboard

Once training starts, the dashboard shows real-time charts powered by WebSockets. You will see:

1. **Training Loss Curve**
   Shows the loss value at each training step. A healthy curve drops steeply then flattens.
2. **Learning Rate Schedule**
   Visualizes the warmup and decay phases of your LR scheduler.
3. **GPU Utilization**
   Shows CUDA memory usage and compute utilization percentage.
4. **Step Counter**
   Tracks total steps completed vs. target, with estimated time remaining.

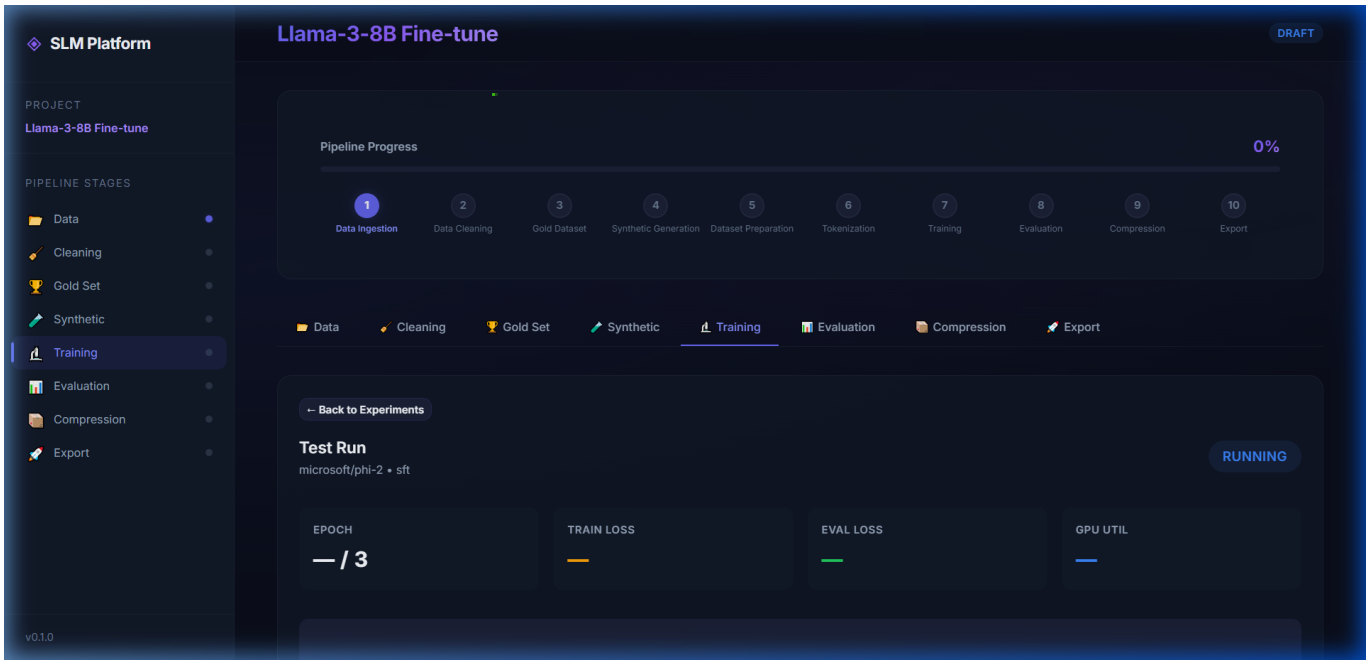

Figure 6.2 -- Live training telemetry with loss curves and GPU metrics

> **TIP**
> If loss plateaus early, try increasing the learning rate or LoRA rank. If loss spikes, reduce the learning rate by 50%.

Chapter 7

Evaluation & LLM-as-a-Judge

The Evaluation module provides both traditional metric-based evaluation and advanced LLM-as-a-Judge benchmarking. A strong Judge model (e.g. Llama-3-70B) scores your SLM's outputs and provides technical rationale.

Running Standard Evaluation

1. **Navigate to 'Evaluation'**
   Click the Evaluation tab in the sidebar.
2. **Select an Experiment**
   Choose a completed training experiment from the list.
3. **Run Evaluation**
   Click 'Run Evaluation'. The system tests against your gold set and computes metrics (BLEU, ROUGE, exact match).

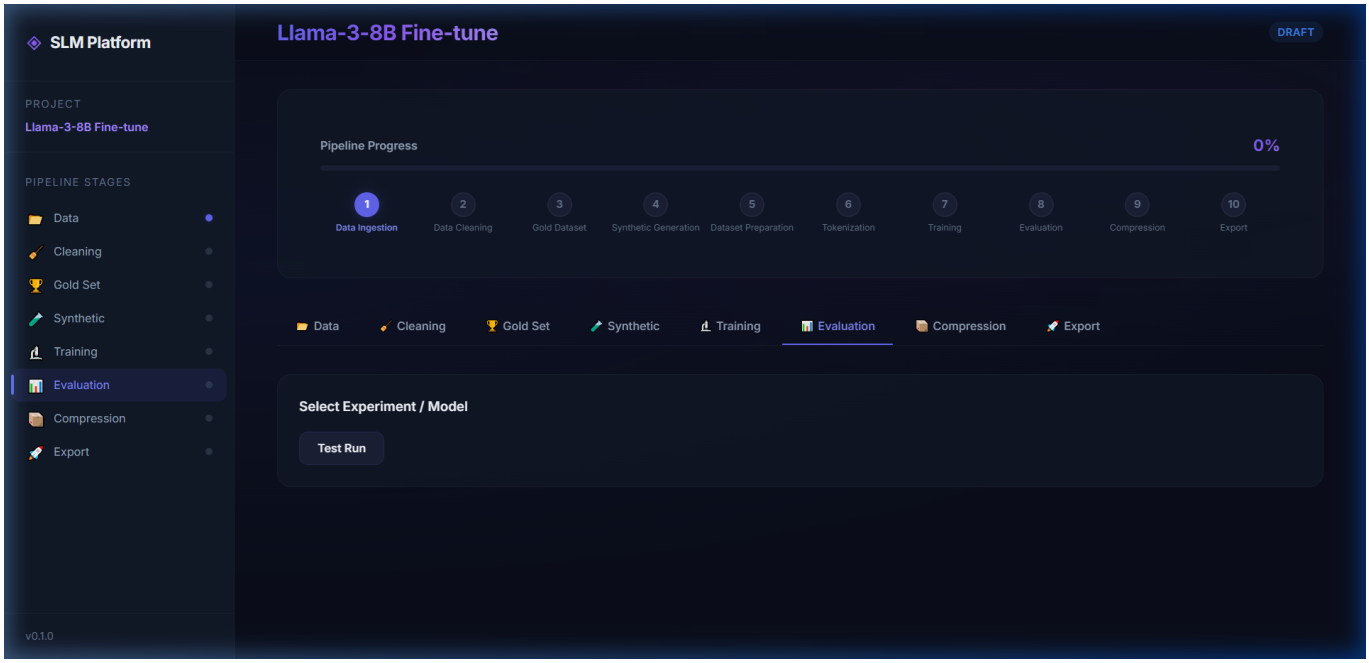

Figure 7.1 -- Evaluation panel with experiment selection

LLM-as-a-Judge Benchmarking

1. **Click '+ Run LLM Benchmark'**

   Opens the benchmark configuration form.

2. **Select Judge Model**

   Choose a strong evaluator (default: meta-llama/Meta-Llama-3-70B-Instruct).

3. **Select Benchmark Suite**

   Choose from MMLU-Subset, HumanEval, GSM8K, or custom datasets.

4. **Click 'Run Benchmark'**

   The Judge model evaluates each prediction on a 1-5 scale.

5. **Review Radar Chart**

   A skill radar visualizes pass rates across different capability domains (Factuality, Logic, Safety, Tone, Helpfulness).

6. **Inspect Side-by-Side Arena**

   Compare Base Model vs. Trained SLM outputs with the Judge's score and rationale for each prompt.

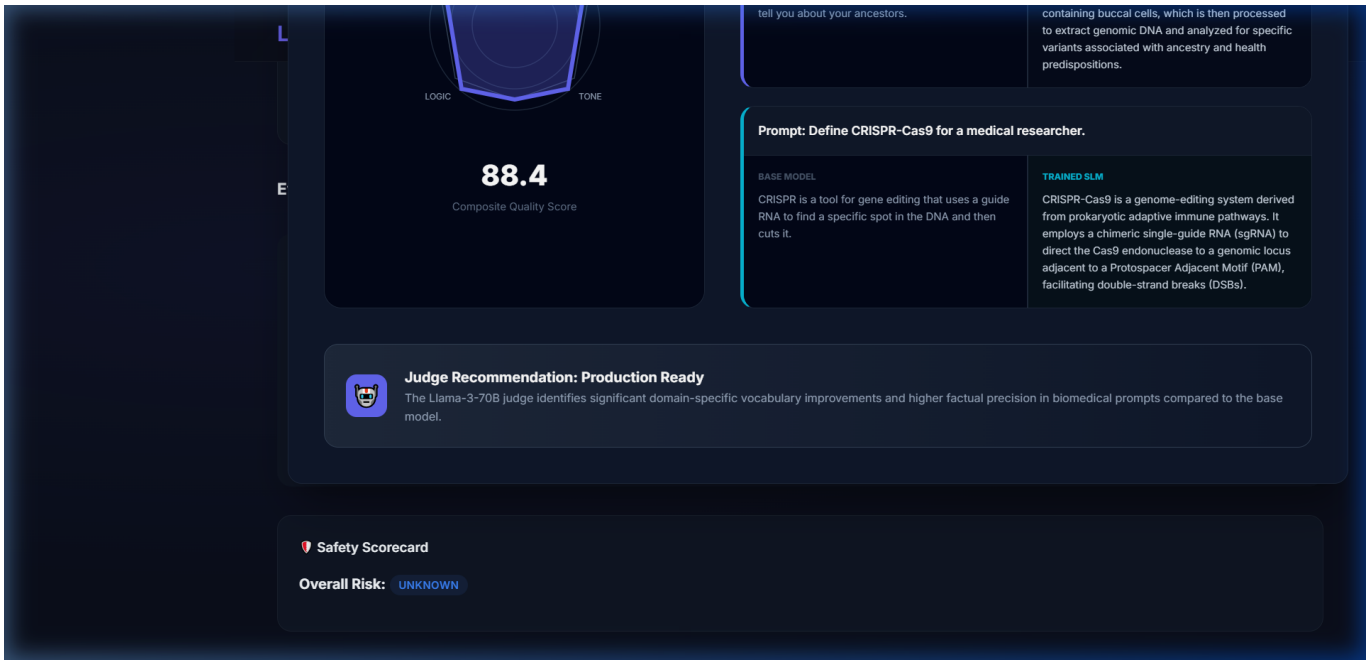

Figure 7.2 -- LLM-as-a-Judge results: Radar analytics and side-by-side comparison arena

> **TIP**
>
> The side-by-side arena is particularly useful for identifying specific failure modes. Look for prompts where your SLM scores below 3.

Chapter 8

Compression & Quantization

After training, compress the model for faster inference and smaller deployment footprint. The platform supports multiple quantization strategies.

Step-by-Step

1. **Navigate to 'Compression'**
   Click the Compression tab in the sidebar.
2. **Select Quantization Level**
   Choose 4-bit (smallest, slight quality loss) or 8-bit (balanced).
3. **Select Method**
   GPTQ for GPU inference, AWQ for optimized deployment, or BitsAndBytes for on-the-fly quantization.
4. **Merge LoRA Weights**
   Check 'Merge before quantization' to bake the LoRA adapters into the base weights.
5. **Run Compression**
   Click compress. The output will be a standalone model ready for export.

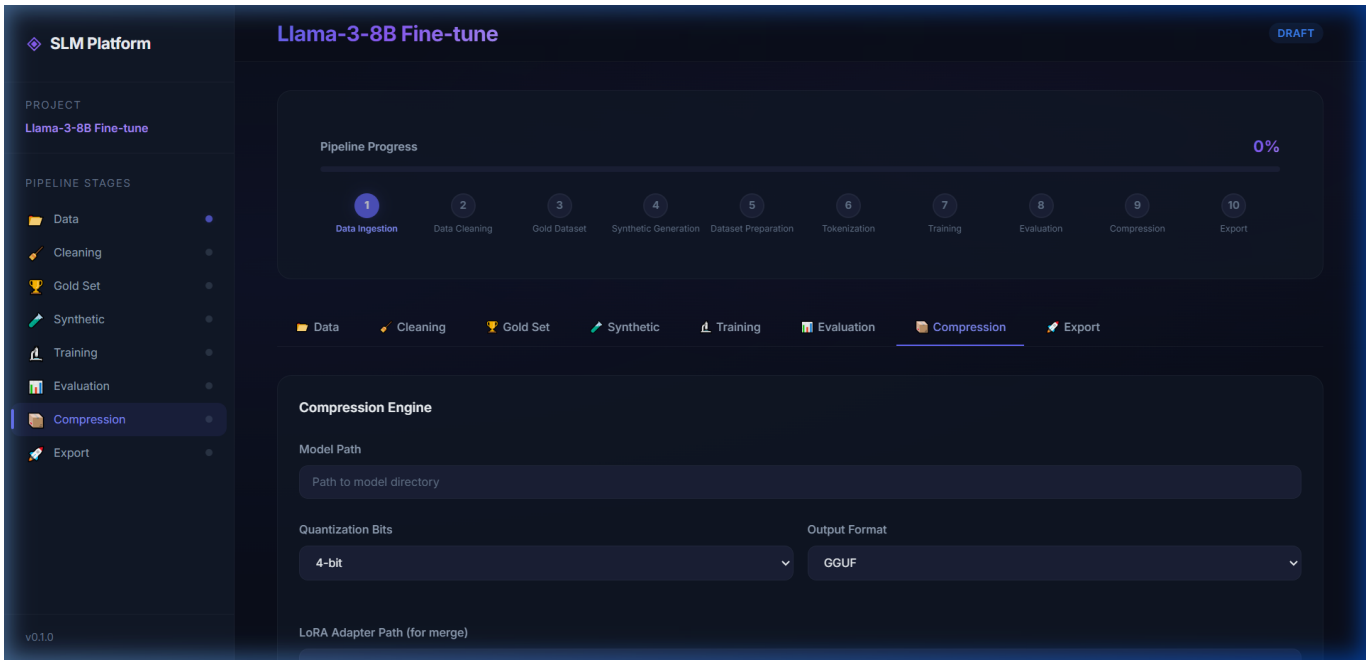

Figure 8.1 -- Compression panel with quantization options

> **TIP**
> 4-bit GPTQ models run 3-4x faster than FP16 on consumer GPUs with minimal quality loss. Always benchmark after quantization.

Chapter 9

Export & Deployment

The final stage packages your fine-tuned, compressed model into a deployment-ready format.

Step-by-Step

1. **Navigate to 'Export'**
   Click the Export tab in the sidebar.
2. **Select Export Format**

- GGUF: For local inference with llama.cpp or Ollama.
- ONNX: For cross-platform deployment with ONNX Runtime.
- Safetensors: Standard HuggingFace format for cloud serving.
- Docker Container: Auto-generates a Dockerfile with inference server.

3. **Configure Metadata**
   Set model name, description, and license for the model card.
4. **Push to HuggingFace Hub (Optional)**
   Enter your HF token and repository name to publish directly.
5. **Download Locally**
   Click 'Export' to download the packaged model to your local machine.

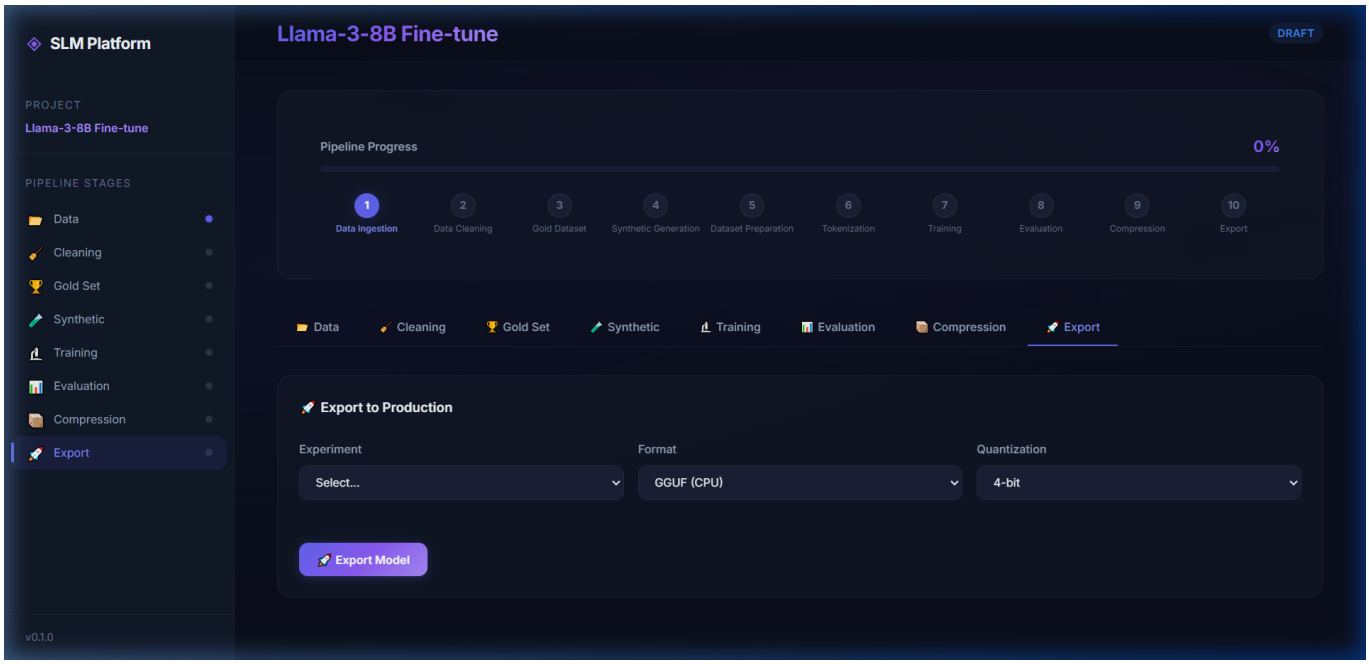

Figure 9.1 -- Export panel with format selection and Hub publishing

Chapter A

Architecture Reference

System Architecture

Frontend: React 18 + Vite + TypeScript + Zustand (state) + Recharts (visualization)

Backend: Python 3.11 + FastAPI + SQLAlchemy (async) + PyTorch + HuggingFace

Database: SQLite (development) / PostgreSQL (production)

Compute: NVIDIA CUDA with Flash Attention 2, bitsandbytes, PEFT

Key API Endpoints

POST	`/projects/{id}/ingestion/upload-batch`	Upload local files
POST	`/projects/{id}/ingestion/import-remote`	Import from HuggingFace/Kaggle/URL
POST	`/projects/{id}/cleaning/run`	Run data cleaning pipeline
POST	`/projects/{id}/training/start`	Launch training job
GET	`/experiments/{id}/metrics/ws`	WebSocket for live metrics
POST	`/projects/{id}/evaluation/llm-judge`	Run LLM-as-a-Judge benchmark
POST	`/projects/{id}/export/run`	Export model to specified format

Environment Variables

DATA_DIR	Path to data storage directory
DATABASE_URL	SQLAlchemy connection string
KAGGLE_USERNAME	Kaggle API username (for Kaggle connector)
KAGGLE_KEY	Kaggle API key
HF_TOKEN	HuggingFace access token (for gated models and Hub push)
CUDA_VISIBLE_DEVICES	Select which GPUs to use (e.g. 0,1)